



Automated Red Teaming for Generative Models: A Survey

Anonymous Author(s)^{1,*}

1. Anonymous Institution

Received month dd, yyyy; accepted month dd, yyyy

E-mail: anonymous@example.com. * These authors contributed equally to this work.

© Higher Education Press 2026

Abstract

The rapid deployment of generative models has created a need for automated red-teaming methods that discover failures beyond manually authored test cases. This survey focuses on automated red-teaming systems that generate, execute, and evaluate adversarial or safety-relevant interactions with generative models and model-based applications. We study automated red teaming across large language models, vision-language models, text-to-image and text-to-video systems, retrieval-augmented applications, and tool-using agents. We organize the literature around a closed-loop pipeline that connects risk objectives, test-case generation, interaction and execution, outcome judgment, failure triage, and mitigation. Within this pipeline, we unify red-teaming strategies by their search space, optimization signal, access assumptions, interaction horizon, and modality. We then compare evaluation metrics, benchmarks, automated judges, human-in-the-loop protocols, and reporting practices, with particular attention to attack success, harm severity, coverage, transferability, cost, and over-refusal. Finally, we identify open problems in long-horizon interaction, multimodal and multilingual testing, reliable evaluation, adaptive coverage, reproducibility, and the responsible use of automated red-teaming systems. The survey is intended to provide a common vocabulary and a practical roadmap for building scalable, evidence-based automated red-teaming evaluations.

Key words

automated red teaming; generative models; large language models; vision-language models; text-to-image generation; AI safety; adversarial evaluation; jailbreaks

■ 1 Introduction

1.1 Motivation: Problem Scope

- Why red teaming matters.

Generative models can fail under inputs that are rare, indirect, context-dependent, or outside the distribution represented by standard safety datasets. Red teaming treats safety assessment as proactive discovery: a tester constructs situations that may expose harmful compliance, privacy leakage, policy evasion, unsafe tool use, or other undesirable behavior before deployment.

- Why automation matters.

Manual red teaming provides valuable expertise but is expensive, difficult to reproduce, and limited in coverage. Automated red teaming extends this process by generating test cases, searching through interaction states, executing attacks, judging outputs, and retaining failures as regression tests. The central challenge is not merely to maximize attack success, but to discover diverse, relevant, and verifiable failure modes.

1.2 Survey Positioning: Research Questions

- Scope.

This survey focuses on automated or partially automated red-teaming systems for generative models. LLMs, VLMs/MLLMs, T2I/T2V systems, RAG applications, and tool-using agents are treated as target

systems. General alignment, generic adversarial examples, and static safety benchmarks are included only when they support an automated red-teaming component.

- Research questions.

The survey is organized around five questions: how red-team objectives are specified; how test cases are generated and searched; how interactions and failures are evaluated; how methods transfer across models, modalities, and applications; and how discovered failures improve subsequent mitigation and regression testing.

- Relation to prior surveys.

Prior surveys provide complementary views of generative-model red teaming, LLM security, attack evaluation, practical red-teaming systems, and multimodal safety [1–5]. We use these works as field-level anchors, then reorganize the evidence around automation boundaries, closed-loop feedback, and target-system transfer.

1.3 Contributions: Article Roadmap

- Contributions.

The survey makes four contributions:

- a closed-loop framework that connects risk specification, attack generation, execution, judgment, triage, and mitigation;
- a taxonomy of automated generation methods based on search space, feedback, access, interaction, and modality;

- a comparison of evaluators, metrics, benchmarks, and reporting protocols; and
- a research agenda for coverage, reliability, multimodality, long-horizon testing, and responsible use.

• Article roadmap.

Section 2 describes the survey protocol. Section 3 defines the threat model and closed-loop framework. Section 4 reviews automated generation and search. Section 5 discusses judgment, metrics, and benchmarks. Section 6 organizes evidence by target system. Section 7 connects failures to mitigation. Section 8 covers tools, human expertise, and responsible use. Section 9 presents open problems.

■ 2 Survey Methodology: Scope

2.1 Literature Protocol: Evidence Base

• Retrieval strategy.

Report the databases, search engines, venues, time range, and query terms used to collect papers. Search terms should include automated red teaming, adversarial search, jailbreak generation, safety evaluation, prompt injection, MLLM safety, T2I safety, agent security, and red-team benchmark.

• Screening procedure.

Describe deduplication, title–abstract screening, full-text screening, citation chaining, and the replacement of preprints by published versions when available.

2.2 Inclusion Criteria: Coding Schema

• Inclusion rule.

Include a paper when automation contributes to risk specification, test generation, adversarial search, interaction, outcome judgment, benchmark construction, failure triage, or mitigation using red-team traces. Separate direct red-teaming papers from supporting safety, jailbreak, and agent-security papers.

• Paper-level coding.

Annotate target system, risk, attacker access, generation mechanism, interaction horizon, feedback signal, evaluator, benchmark, metric, compute cost, and outcome. Use this schema to build comparison tables rather than summarizing papers as an unrelated sequence.

2.3 Survey Limitations: Coverage Bias

• Evidence limitations.

State the cutoff date, language restrictions, treatment of unpublished work, benchmark bias, closed-model access limitations, and the risk that rapidly changing attack methods become obsolete.

■ 3 Unified Automated Red-Teaming Framework

3.1 Definitions: System Boundaries

• Automated red teaming.

Define automated red teaming as goal-directed discovery of undesirable behavior in which one or more stages of test generation, selection, interaction, judgment, or iteration are automated under an explicit threat model.

• Target system boundary.

Separate the target model, safety layer, orchestration layer, retrieval layer, memory, tools, and deployed application. A discovered failure should be attributed to the smallest component or interaction responsible for it.

3.2 Threat Model: Attack Conditions

• Attacker access.

Distinguish black-box, gray-box, white-box, and application-level access. Record query limits, model knowledge, system-prompt visibility, gradient access, activation access, control over retrieved content, and control over tools or memory.

• Risk objective.

Classify objectives into harmful compliance, privacy leakage, prompt injection, policy evasion, misinformation, bias, copyright exposure, unsafe tool use, reliability failure, and over-refusal.

• Interaction horizon.

Separate single-turn, iterative, multi-turn, stateful, multi-agent, and long-horizon settings. The horizon determines whether the attack optimizes a response, a conversation trajectory, or an external action.

3.3 Closed-Loop Pipeline: Evaluation Cycle

• Risk specification.

Translate a policy, threat model, harm taxonomy, or application requirement into a test objective and an observable failure condition.

• Test generation.

Construct an initial population of prompts, scenarios, images, videos, retrieved documents, tool outputs, or multi-turn plans.

• Search: Interaction.

Select, mutate, optimize, or compose test cases while interacting with the target system. The search state may include prompts, conversation history, tool state, model feedback, and prior failures.

• Outcome judgment.

Determine whether the target behavior satisfies the failure condition using rules, classifiers, reward models, LLM judges, VLM judges, environments, or human annotators.

• Triage: Regression.

Cluster failures, estimate severity and novelty, remove duplicates, record reproducible attack traces, and convert confirmed failures into held-out regression tests.

■ 4 Automated Generation: Search Strategies

4.1 Scenario Construction: Initial Test Cases

• Seed prompts.

Review expert prompts, policy-derived scenarios, safety datasets, attack libraries, synthetic harmful-intent descriptions, and application-specific user stories.

• Scenario expansion.

Cover semantic paraphrasing, translation, role-play, encoding, formatting, contextual embedding, instruction indirection, and risk-conditioned generation.

- Diversity control.

Compare novelty penalties, coverage constraints, population methods, quality-diversity search, clustering, and explicit demographic, cultural, or geographic balancing.

4.2 Model-Based Generation: Attacker Models

- Attacker language models.

Review zero-shot generation, proposer–judge loops, attacker–target interaction, model-written evaluations, and attacker fine-tuning.

- Reverse generation.

Discuss methods that begin from a harmful output, policy violation, or target behavior and search backward for an input that elicits it.

- Multi-agent generation.

Separate attacker, critic, judge, planner, and target roles. Analyze whether role separation improves exploration or introduces correlated errors.

4.3 Algorithmic Search: Optimization Signals

- Fuzzing: Mutation.

Cover seed selection, mutation operators, random search, evolutionary search, and black-box feedback.

- Reinforcement learning.

Review attacker training with reward signals from toxicity classifiers, safety judges, target compliance, novelty, or environment outcomes.

- Gradient-Level: Token-Level: Activation-Level Optimization.

Describe white-box optimization of suffixes, tokens, continuous embeddings, image perturbations, or internal representations. Compare access requirements, transferability, realism, and cost.

4.4 Adaptive Interaction: Stateful Search

- Iterative refinement.

Analyze attacks that learn from refusals, partial compliance, judge feedback, or changes in target behavior.

- Multi-turn search.

Cover trust building, context shaping, refusal recovery, conversational state, memory persistence, and long-horizon planning.

- Application-level search.

Extend the search state to retrieved documents, web pages, tool outputs, user permissions, external state, and action consequences.

■ 5 Evaluation: Judgment: Benchmarking

5.1 Failure Definition: Success Conditions

- Behavioral success.

Distinguish policy violation, harmful content, unsafe action, privacy leakage, misinformation, reliability failure, and evaluator disagreement. Every benchmark should define the target behavior before reporting attack success.

- Severity: Usefulness.

Measure not only whether a guardrail was bypassed, but also whether the response contains actionable, relevant, and sufficiently complete harmful information or performs an unsafe action.

5.2 Automated Judges: Human Agreement

- Judge families.

Compare keyword rules, lexical matching, safety classifiers, reward models, LLM-as-judge, VLM-as-judge, structured rubrics, environment success signals, and human review.

- Judge reliability.

Report calibration, inter-rater agreement, judge disagreement, false positives, false negatives, judge hacking, and agreement with human judgments.

5.3 Metrics: Experimental Comparability

- Attack effectiveness.

Cover attack success rate, harmfulness score, refusal rate, compliance quality, and task-level success.

- Search quality.

Cover coverage, diversity, novelty, marginal discovery rate, transferability, robustness, query efficiency, latency, and compute cost.

- Safety–utility balance.

Report over-refusal, benign-task utility, calibration, safe completion quality, and the trade-off between harmfulness reduction and helpfulness.

5.4 Benchmarks: Protocol Design

- Static datasets.

Compare behavior sets, harmful instructions, refusal suites, adversarial examples, multimodal pairs, and application-specific test collections.

- Dynamic environments.

Discuss extensible harnesses, agent environments, tool-use simulators, external-state tasks, adaptive attacks, and continual benchmark updates.

- Reproducibility checklist.

Require model version, system prompt, access condition, sampling parameters, query budget, random seeds, judge version, annotation protocol, compute cost, attack artifacts, and release policy.

■ 6 Automated Red Teaming Across Generative Models

6.1 Large Language Models: Text Interaction

- Prompt-level attacks.

Review semantic jailbreaks, mutation-based fuzzing, role-play, encoding, multilingual prompts, prompt injection, and refusal manipulation.

- Token-level attacks.

Review adversarial suffixes, gradient-guided optimization, transfer attacks, activation steering, and access-dependent evaluation.

- Conversation-level attacks.

Analyze multi-turn persuasion, context contamination, many-shot behavior, memory persistence, and stateful safety drift.

6.2 VLMs: MLLMs: Cross-Modal Inputs

- Visual attack surfaces.

Organize attacks by text-only inputs, malicious images, visual instructions, OCR, image–text mismatch, adversarial perturbations, and jointly optimized inputs.

- Cross-modal evaluation.

Track whether the failure originates in the vision encoder, fusion module, language model, safety layer, or modality interaction.

6.3 T2I: T2V: Diffusion Systems

- Text-to-image testing.

Cover unsafe prompt generation, safety-filter bypass, semantic obfuscation, image-space perturbation, prompt-image mismatch, and benign prompts that produce harmful images.

- Text-to-video testing.

Extend the taxonomy to temporal consistency, unsafe actions, identity and privacy risks, audio or video conditioning, and harms that emerge across frames.

- Image-aware judgment.

Require image or video evaluators and targeted human validation rather than text-only safety labels.

6.4 Agents: RAG: Tool-Use

- Indirect prompt injection.

Cover malicious instructions embedded in retrieved documents, web pages, files, tool outputs, and external data.

- Tool misuse.

Evaluate privilege escalation, unsafe planning, unauthorized actions, data exfiltration, and failures caused by tool output interpretation.

- Environment-level success.

Measure action consequences, security-property violations, task completion, reversibility, and user impact rather than textual harmfulness alone.

6.5 Multilingual Testing: Cultural Context

- Language coverage.

Discuss low-resource languages, code-switching, translation, dialect, and language-specific safety gaps.

- Cultural coverage.

Discuss regional norms, demographic context, participatory testing, and the risk that automated generation reproduces cultural bias.

■ 7 Failure Remediation: Safety Improvement

7.1 From Red-Team Traces to Safety Tuning

- Remediation as a Closed Loop.

Failure discovery need not be the terminal stage of automated red teaming. Red-team traces can expose concrete conditions under which a target model produces harmful or otherwise undesirable behavior, creating evidence for subsequent safety tuning. Ganguli et al. frame red teaming as a process that discovers, measures, and attempts to reduce harmful outputs, and release 38,961 red-team attacks for further analysis [6]. Perez et al. demonstrate that one language model can generate test cases for another and uncover tens of thousands of offensive replies across multiple failure types [7]. Their work primarily establishes scalable failure discovery and characterization, rather than direct remediation of the target model. MART connects these stages by generating adversarial prompts, using them in target-model safety fine-tuning, and launching new attacks against the updated target [8]. This progression turns red-team evidence into a potential input to

model improvement, motivating the conversion of confirmed failures into training supervision.

- From Failure Traces to Training Supervision.

Recent systems reuse automatically generated adversarial prompts as safety-training data rather than treating them only as test cases. MART directly fine-tunes the target model using safety-aligned data constructed around adversarially elicited prompts before launching another attack round [8]. Related work reuses diverse generated attacks for safety tuning and evaluates the resulting models against other reinforcement-learning-based red-teamers [9], while DiveR-CT links the diversity and composition of collected attacks to downstream blue-team tuning [10]. Taken together, these studies suggest that, from a closed-loop red-teaming perspective, the common element is not a particular optimization procedure but the reuse of failure-inducing inputs to construct subsequent safety supervision. Once these traces are reused for model updates, the red-team generator begins to shape the model's evolving safety-training distribution.

- Targeted Repair and the Safety-Utility Trade-off.

Targeted safety supervision can substantially improve measured safety, but remediation should be evaluated jointly with its effects on benign behavior. Safety-Tuned LLaMAs reports that adding roughly 3% safety examples, corresponding to a few hundred demonstrations, substantially improves safety without significant degradation on standard capability and helpfulness benchmarks [11]. The same study finds exaggerated safety under more extensive safety tuning, including refusals on prompts that are safe but superficially resemble unsafe requests. RASS addresses this boundary problem by automatically generating and selecting prompts near the safety decision boundary for targeted over-refusal mitigation [12]. XSTest provides a complementary evaluation protocol with 250 safe prompts and 200 unsafe contrasts for measuring exaggerated safety [13]. These findings show that safety improvement cannot be measured by harmful-query refusal alone. The resulting safety-utility trade-off motivates a shift from one-shot safety tuning toward training procedures in which new failures are repeatedly generated against the updated model.

- From One-Shot Repair to Adaptive Adversarial Training.

A fixed set of failure traces may support an initial repair, but it cannot by itself reveal how vulnerabilities change after the target model is updated. Automated red teaming becomes part of the training-data generation process when new attacks are generated against the revised model and fed back into subsequent updates. MART provides the clearest example: an adversarial model generates prompts that elicit unsafe responses, the target model is safety-fine-tuned using safety-aligned data constructed around the resulting adversarial prompts, and the attacker then searches against the updated target in the next round [8]. MART reports up to an 84.7% reduction in violation rate after four rounds, while helpfulness on non-adversarial prompts remains stable. HarmBench provides a complementary adversarial-training setting through R2D2, which maintains persistent test cases that are dynamically optimized during training and evaluates attack-defense performance across a standardized collection of methods and models [14]. Learning Diverse Attacks and DiveR-CT provide supporting

evidence that the diversity and composition of generated attacks affect downstream safety tuning, although they do not implement the same attacker–defender loop as MART [9, 10]. Once generated attacks are reused for model updates, red-team search helps define the model’s evolving safety-training distribution.

- **Robustness to Training Attacks Is Not General Robustness.**

If automated attacks shape the safety-training distribution, evaluation must also ask what happens outside that distribution. Jailbroken identifies competing objectives and mismatched generalization as two failure modes through which safety training can remain vulnerable despite extensive safety training [15]. Direct counterevidence comes from Simple Adaptive Attacks, which reports 100% attack success on several tested safety-aligned models, including R2D2, when the attack is adapted to the target model [16]. A different distribution shift appears in the past-tense reformulation study: on GPT-4o, the reported success rate increases from 1% for direct requests to 88% after 20 past-tense reformulation attempts [17]. Explicitly adding past-tense examples can improve defense against that specific reformulation, but this result reinforces rather than resolves the generalization problem. Safety Reasoning with Guidelines similarly reports that direct refusal training can rely on superficial mappings under out-of-distribution attacks, while guideline-based reasoning improves the reported OOD performance [18]. Together, these findings indicate that remediation may fail to generalize beyond the failure patterns represented during training.

- **Evaluating Remediation Beyond the Training Distribution.**

Training-time remediation should therefore be evaluated along three dimensions: held-out harmful behaviors, unseen or adaptive attack strategies, and benign utility. HarmBench standardizes comparisons across 18 red-teaming methods and 33 target models and defenses, while also showing that no tested defense is uniformly effective [14]. JailbreakBench complements this setting with an open collection of attack artifacts, 100 behaviors, and standardized threat-model and scoring protocols [19]. These benchmarks support reproducible comparison, but neither should be treated as a complete measure of deployment risk. Benign behavior requires separate evaluation. XSTest pairs 250 safe prompts with 200 unsafe contrasts to measure exaggerated safety [13], while Safety-Tuned LLaMAs reports that stronger safety tuning can produce refusals on superficially similar safe prompts [11]. Past-tense reformulations and adaptive attacks provide additional tests beyond the remediation-time distribution [16, 17]. No single benchmark covers all three dimensions. Training-time remediation should be viewed as expanding the region over which safety behavior generalizes, rather than merely eliminating the failures used as training examples. Retaining confirmed failures as reusable regression cases is discussed in Section 7.4.

7.2 From Safety Judgment to Runtime Control

- **Runtime intervention.**

Training-time mitigation changes model behavior through additional optimization, whereas inference-time mitigation turns safety judgments into runtime controls while leaving the target parameters fixed

or largely unchanged. In the closed-loop view of automated red teaming, evaluator outputs that were previously used to identify failures can instead become runtime signals for filtering or controlling generation. This shift connects the judge, feedback, and generation components of the red-team loop to deployment-time guardrails.

- **Evaluator-to-guard transfer.**

A first reuse pattern moves the red-team judge into the deployment path. WildGuard unifies prompt harmfulness detection, response safety-risk detection, and refusal detection for language models, while Llama Guard and ShieldGemma provide related input and output moderation capabilities [20–22]. Such evaluator-like models can be placed before the target to inspect user inputs, after the target to inspect generated responses, or at both locations. NeMo Guardrails makes this connection explicit by linking moderation decisions to programmable runtime rails [23]. Once placed in the deployment path, the evaluator’s prediction becomes an intervention signal rather than merely an offline assessment. This transition is not automatic, however. The calibration study of LLM-based guard models reports overconfidence, jailbreak-induced miscalibration, and variation across response models [24]. Reliable deployment therefore requires evaluating calibration and operating thresholds in addition to aggregate classification accuracy.

- **Locations of runtime control.**

Safety judgments can enter the runtime pipeline at different locations. Before generation, a system can analyze intent, rewrite a prompt, add policy guidance, or route the request through a restricted flow. Intention Analysis uses a two-stage inference-only procedure in which the model first analyzes the essential intent of an input and then conditions the final response on that analysis [25]. NeMo Guardrails provides a broader framework for connecting such checks to runtime controls [23]. During generation, the safety signal can modify token probabilities or internal representations. SafeDecoding adjusts decoding probabilities using safety-aware token-level information, whereas Jailbreak Antidote steers a sparse subset of hidden states along a safety direction during inference [26, 27]. After generation, output-side guards can classify generated responses, while runtime frameworks can use these judgments to block, route, or otherwise control unsafe outputs [20, 21, 23]. These mechanisms differ in model access, latency, controllability, and dependence on the target architecture, but they share a shift from passive evaluation toward active runtime control.

- **Feedback-conditioned defensive generation.**

A similar feedback structure appears in automated attack refinement. Automated attackers use target or judge feedback to update subsequent attacks, whereas inference-time defenses can use safety feedback to condition ongoing or subsequent generation. SafeDecoding and Immune illustrate this pattern through token-level safety signals and safety rewards, respectively [26, 28]. Intention Analysis uses an intermediate interpretation of attacker intent to control a subsequent response pass, while Reasoning-to-Defend illustrates a related strategy in which safety reflections and safety-pivot tokens influence response generation [25, 29]. The analogy is structural rather than algorithmic: the two processes use feedback-conditioned generation toward opposite objectives, and several defensive methods rely on target-specific decoding or

trained behaviors rather than a model-agnostic judge-and-regenerate wrapper.

- Cross-modal safety signals.

Multimodal systems make the location of the safety signal less obvious. A text-only guard may miss a malicious image paired with an innocuous-looking instruction, a harmful meaning created by image-text interaction, or an attack encoded in a modality-specific representation. One line of work addresses image-side adversarial perturbations before they reach the target model. CIDER, for example, uses cross-modal similarity to detect maliciously perturbed image inputs and is designed to operate independently of the target MLLM [30]. Multimodal moderation extends the judge itself beyond text: Llama Guard 3 Vision classifies image-understanding conversations, while MULTIGUARD introduces OMNIGUARD for safety moderation across language, image, and audio inputs [31, 32]. Immune represents a different intervention point by using a multimodal safety reward to guide decoding [28]. These methods suggest three complementary guard locations: cross-modal input detection, multimodal moderation, and decoding-time safety guidance. They do not imply that prompt-side detection provides complete coverage of generated images, videos, or audio.

- Generative media and audio.

For generative media, input-side filtering is insufficient when risk emerges during generation or only becomes observable in the generated output. SAFREE represents generation-time intervention for text-to-image and text-to-video systems by detecting toxic concepts in the text-embedding space, steering generation away from unsafe subspaces, and applying self-validation during diffusion sampling [33]. T2VShield extends protection across prompt processing and generated video by combining prompt rewriting with local and global analysis of temporal and cross-modal inconsistencies [34]. The need for output- and temporal-aware protection is reinforced from the attack side by TEAR. TEAR constructs textually innocuous prompts that exploit temporal event sequencing to elicit policy-violating video content and reports high attack success across open-source and commercial T2V systems [35]. TEAR is not an inference-time defense, but it shows why prompt-level or static-content filters can miss harms that emerge through temporal composition.

Audio exposes a related but different gap because safety-relevant information can reside in both semantic content and waveform-level signals. SALMONN-Guard jointly inspects speech, audio, and text to intercept compositional attacks before the target model is queried [36]. AudioGuard further separates waveform-level and semantic content risks and combines specialized decisions under different threat models [37]. Among the audio defenses reviewed here, existing methods concentrate on input-side interception, whereas generated-audio moderation remains comparatively underrepresented. Across text, image, video, and audio systems, the appropriate guard location therefore follows the location of the risk signal: textual intent, visual content, cross-modal interaction, generated output, acoustic structure, or temporal evolution.

- Guardrails as an attack surface.

Guardrails consequently become part of the attack surface. Calibration failures can undermine threshold-based decisions [24], while multimodal and generative-media studies show that harmful behavior can arise from modality interactions or become observable only in generated outputs [31, 34, 35]. Guardrail evaluation should therefore extend beyond accuracy on a fixed attack set to include calibration, benign utility, runtime cost, and robustness to adaptive or transferred attacks. Inference-time guardrails do not eliminate the need for automated red teaming; they expand its target from the base model to the full runtime safety pipeline. Confirmed guardrail bypasses can then become regression cases for continuous red teaming in Section 7.4.

7.3 Application Mitigation: System Controls

- Model and system safety.

Model-level safety does not by itself establish system-level safety. The safeguards discussed in Sections 7.1 and 7.2 primarily reduce the probability that a model produces an unsafe response or follows a harmful instruction. Once the model is connected to retrieval, tools, persistent state, and external services, however, a failure can also alter control flow, exercise an unauthorized capability, disclose data, or change an external state. AgentDojo makes this distinction operational by evaluating agents against security properties and tool consequences rather than only inspecting their final text [38]. A central application-level requirement is therefore to separate information that the model may read from authority that can redefine the task. In indirect prompt-injection settings, the system-level failure is not merely exposure to untrusted content, but allowing that content to acquire control authority over application behavior. CaMeL implements this separation by extracting control and data flows from the trusted query and restricting the influence of untrusted retrieved content on program flow; it further uses capabilities to constrain unauthorized information flows [39]. These mechanisms preserve a distinction between data that informs execution and instructions that control it. External data may inform the task without being allowed to redefine it.

- Independent system controls.

Even if an injection crosses this trust boundary and manipulates the model's behavior, an independent boundary around execution can still limit the resulting effect. The first mechanism is isolation from real effects. IsolateGPT separates application execution and mediates interactions between applications and the surrounding system through controlled interfaces and permissions, providing a concrete example of effect isolation [40]. The second mechanism is authorization before execution. The Task Shield checks whether instructions and proposed tool calls contribute to the user's specified goals, while ToolSafe detects unsafe tool invocations before execution and feeds safety judgments back into the agent's reasoning [41, 42]. Together, these approaches insert independent system controls between model decisions and external effects, through isolation, mediation, or pre-execution authorization. This separation matters because a model decision and a system authorization are different events. A model may decide that it should send an email, read a file, or invoke a service, while an independent

application layer can still check whether the operation serves the user's task and falls within the authority granted to the agent. The model proposes an action; the system determines whether that action is executable. Evidence from coding-agent authorization further cautions against delegating this decision to the model itself: AuthBench reports both omitted permissions needed for a task and grants of unused or sensitive access, and additional inference-time reasoning does not remove the mismatch [43]. These results support enforcing authorization through independent system controls rather than relying on the model alone to determine its own privileges.

- Consequences and recovery.

At the application boundary, failure should therefore be measured by its consequence, not only by the model's decision. Application-level evaluation should track unauthorized actions, data exfiltration, and protected-state changes rather than textual behavior alone. Agent-Dojo evaluates these questions in realistic tool-using workflows, while ToolEmu uses an LM-emulated sandbox to test high-stakes tool-use failures before real effects occur [38, 44]. The literature reviewed here is substantially more developed around preventing or isolating unsafe actions than around recovery after external side effects. Once an agent has sent an email, modified a permission, or written to an external service, the effect may no longer be fully under the application's control. Recovery is consequently harder than blocking a proposed action before execution, especially when the side effect is irreversible or crosses service boundaries. Early transactional designs such as Cordon explore staged effects and commit/rollback boundaries, but evidence for general recovery remains limited [45].

- System-level red teaming.

A system-level red-team test should ask whether untrusted data acquired control authority, whether a compromised model obtained unauthorized capabilities, and whether prohibited effects were executed or contained. Application-level mitigation therefore extends automated red teaming from testing model behavior to testing whether application-level security properties survive model failure. Adaptive attacks against system-control mechanisms remain under-tested, while post-execution recovery from durable side effects remains comparatively immature.

7.4 Regression Testing: Continuous Red Teaming

- Failure promotion.

Continuous red teaming turns a one-time remediation result into a replayable safety workflow. A raw attack success is not sufficient for reliable regression testing. We argue that a replayable failure case should preserve the observed behavior, target configuration, evaluator, threat condition, and reproduction context. Additional metadata such as severity and deduplication status can support triage and prioritization. HarmBench provides a standardized evaluation framework with reusable test cases, target models, defenses, and attack methods, while JailbreakBench provides an open collection of behaviors, attack artifacts, wrappers, and scoring protocols [14, 19]. These resources motivate reusable evaluation artifacts, but neither should be treated as a complete software-style regression system. In particular, retaining a

successful prompt without recording the observed behavior, evaluator, threat condition, and reproduction context preserves an input rather than a replayable failure case.

- Version-aware replay.

Regression replay should therefore be version-aware. A previously observed failure may disappear, recur, or change in severity as the model or its mitigation layers are updated. The GPT-4 System Card describes iterative red teaming across model versions and repeated testing as new mitigation and control layers are incorporated [46]. More generally, for model-based applications, the replay target is the evaluated system configuration rather than the model checkpoint alone. Changes in surrounding components such as policies, retrieval, tools, or evaluators can alter the observed outcome; this broader system-level view follows the component boundaries introduced earlier in the survey. A practical case should therefore preserve configuration identifiers and evaluation conditions, so that recurrence, disappearance, and changes in severity can be distinguished from changes caused by an altered judge or execution environment.

- Retained versus renewed testing.

Fixed regression cases and renewed adversarial discovery answer different questions. Retained cases ask whether previously confirmed failures recur after a change; they cannot by themselves show that new attack strategies have not opened a different failure region. AutoRedTeamer addresses the renewal side through memory-guided attack selection and a strategy proposer that continuously discovers and integrates new attack vectors [47]. Jailbreak Distillation renews the evaluation artifact instead: it distills attacks into updateable safety benchmarks and supports rerunning the construction pipeline as new models and attack methods become available [48]. The distinction is consequential: AutoRedTeamer renews the attacker, whereas Jailbreak Distillation renews the benchmark. Passing a retained regression suite provides evidence that the retained failures do not recur under the tested configuration, not evidence of broad safety coverage. Neither paper alone establishes a complete regression workflow for arbitrary model-system changes.

- Continuous red-teaming loop.

A mature remediation loop therefore combines two complementary mechanisms: persistence, which retains verified failures to detect recurrence, and renewal, which updates attacks or evaluation artifacts to probe beyond known failures [46–48]. In this view, mitigation is not the endpoint of red teaming but a state transition in a continuing evaluation loop: retained failure cases are replayed against the mitigated or updated configuration, while renewed search tests whether the repair generalizes beyond the cases that motivated it. Questions about how taxonomies, judges, and evaluation targets can themselves evolve without undermining longitudinal comparability are deferred to Section 9.5.

■ 8 Practice: Tools: Responsible Use

8.1 Frameworks: Evaluation Harnesses

- Framework capabilities.

Evaluation harnesses operationalize automated red-teaming loops by connecting target or model adapters, attack and probe modules, eval-

uators, execution controls, and result logging. Among the harness-oriented systems examined here, this integration layer is not tied to one attack method. *garak*, *PyRIT*, *Inspect AI*, and *promptfoo* illustrate the pattern: each documents mechanisms for binding target systems, applying probes or evaluation logic, managing runs or concurrency, and storing outputs or scores [49–52]. The common abstraction is therefore an executable red-team workflow that can organize targets, evaluation, execution, and records across runs rather than a single jailbreak algorithm.

- Interaction scope.

Harnesses differ most in the scope of interaction they can execute. Evaluation scope ranges from prompt-response probing to modular attacks, multimodal targets, tools, sandboxes, and agent environments. *PyRIT* extends red-team workflows through modular attacks and multimodal targets [50], while *Inspect AI* exposes tasks, solvers, tools, scorers, sandboxes, and execution logs [51]. *AgentDojo* and *CybersecurityBenchmarks* further illustrate evaluation in environments where success depends on tool use or application-level behavior [38,53]. The relevant distinction is not simply the number of supported features, but how much of the target system can be brought inside the executable evaluation loop.

- Benchmark versus harness.

Benchmark and harness are functional roles rather than stages in a replacement sequence. Fixed benchmark artifacts stabilize test cases, behaviors, and scoring protocols for standardized comparison. *MM-SafetyBench* provides this role for multimodal safety evaluation [54]. Extensible harnesses instead provide replaceable targets, attacks, evaluators, and execution components: *PyRIT* supports modular targets, attacks, scorers, memory, and multimodal evaluation [50], while *Inspect AI* supports replaceable model APIs, tasks, solvers, tools, scorers, logs, sandboxing, and re-scoring [51]. Hybrid systems complicate a strict benchmark–harness dichotomy. *HarmBench* combines standardized behaviors and evaluation artifacts with extensible red-team methods and model configurations [14], while *JailbreakBench* combines fixed artifacts, a threat model, scoring functions, model wrappers, and artifact-level attack submissions [19]. The distinction is therefore functional rather than categorical: benchmarks stabilize what is tested, whereas harnesses organize how tests are executed, and modern systems can serve both roles.

- Persistence across runs.

Logging, reproducibility, and regression form increasingly strong levels of evaluation persistence. Logging records what happened in a run, including prompts, outputs, scores, reports, and metadata. Reproducibility preserves enough configuration, model and dataset identifiers, seeds, or artifacts to rerun or compare an evaluation. Here, we use regression testing to refer to deliberately re-executing retained tests or confirmed failures after changes to a model, system, configuration, or defense. The examined systems provide combinations of these mechanisms through saved artifacts, evaluation sets, repeat options, released prompts or completions, and standardized materials [14, 19, 38, 49, 52, 53, 55, 56]. *Inspect AI* provides a particularly explicit example through persistent evaluation logs, saved run configu-

ration, and re-scoring support, which enable repeated analysis of prior evaluations [51]. Explicit regression workflows are less uniformly exposed than logging or rerun support in the systems reviewed here; this observation describes a documentation and workflow gap, not evidence that undocumented systems lack regression capability.

- The automation boundary.

Evaluation harnesses therefore do more than simplify implementation: they shape which parts of the red-team loop can be executed, recorded, compared, and repeated at scale. As evaluations move toward multimodal and agentic systems, this infrastructure increasingly defines the practical scope of automated red teaming. Expanding executable coverage does not remove the need for human expertise in defining risks, validating judges, and interpreting novel failures, which motivates the participatory-testing discussion in Section 8.2.

8.2 Human Expertise: Participatory Testing

- Expanding the risk space.

Human participation can expand the red-team risk space by changing what is searched for, rather than merely adding tests within a fixed objective [57–59]. *Adversarial Nibbler* used crowdsourced red teaming to elicit implicitly adversarial text-to-image prompts, revealing non-obvious, long-tail safety issues and novel attack strategies within that setting [57]. Local knowledge can likewise change the content and interpretation of tests: *Going PLACES* elicited culturally and linguistically situated prompts and annotations in Ghana, Nigeria, Karnataka, and Punjab, while *CulturalBench* combined human-written and human-verified questions across 45 global regions through a process inspired by Human–AI red teaming [58,60]. Human input can also shape the search objective itself. *STAR* operationalizes human-authored, parameterized risk rules, while a participatory study in South Korea shows how lived experience informed stereotype-targeted attack strategies [59,61]. These studies do not imply exhaustive coverage or general superiority over automated discovery. Instead, they show that human participation can introduce risks and contexts omitted by predefined risk objectives [57, 58, 60]. If human participation helps determine what should be tested, automation determines how broadly those specifications can be exercised.

- Scaling human-specified risks.

The literature repeatedly adopts a complementary division of labor in which humans define risks, seeds, contexts, or priorities, while automated systems generate, execute, and screen tests at scale [7, 62–64]. At the specification stage, *AART* encodes application context, policies, harms, populations, regions, and priorities as human-defined recipes that a language model expands into adversarial prompts. Bridging the Scale Gap similarly expands human-authored seeds with human-derived strategies, increasing geographic and demographic coverage without proportional human authoring effort [62,63]. At the scaling stage, Perez et al. use attacker models, classifiers, reinforcement learning, and clustering, but find that optimizing attack effectiveness can reduce diversity; greater automation therefore does not automatically broaden coverage [7]. In operational practice, interviewees describe selecting and ranking harms, inspecting outputs, and handling ambigu-

ous cases while using synthetic generation and automated judges to support scale [64]. Across these systems, automation primarily scales human-defined risks and seeds rather than replacing human specification. The literature supports complementarity more clearly than any fixed division of labor. Scaling test generation also shifts the bottleneck toward judging the resulting failures.

- Human validation of automated judgments.

Automated judges enable large-scale failure screening, while human expertise can enter the judging workflow at targeted points [7,64]. First, human labels can validate an automated judge on a bounded sample before large-scale use, as in Perez et al.'s evaluation of an offensive-language classifier against human annotations [7]. Second, uncertain, novel, or context-sensitive cases can be escalated for human interpretation. Build it Break it Fix it shows that offensive-language detection in dialogue depends critically on conversational context; Going PLACES uses local annotations for culturally situated harms; and Adversarial Nibbler combines machine safety annotations with richer human annotations of harm types and attack styles [57, 58, 65]. Third, disagreement can be adjudicated without treating human judgment as an unquestionable gold standard. STAR gives an arbitrator access to annotator reasoning, DICES preserves individual and demographic ratings alongside expert labels, and D3CODE analyzes culturally structured disagreement rather than collapsing it during collection [59, 66, 67]. These mechanisms support selective human review where context or disagreement matters, while leaving the appropriate escalation criteria largely unresolved [7, 59, 64].

- Limits of human scaling.

Human red teaming is costly and difficult to scale across large or continuously changing test spaces. Practitioner studies report that recruiting suitable evaluators and maintaining manual review throughout the testing pipeline require substantial human effort, making exhaustive human evaluation impractical at scale [64]. Consistent with this constraint, hybrid workflows use automation for high-volume generation and screening while reserving human effort for risk definition and difficult judgments [7, 62, 64]. Human effort is therefore a limited resource within the red-teaming pipeline, making selective involvement more practical than exhaustive manual review.

8.3 From Red-Team Artifacts to Responsible Disclosure

Automated red teaming produces artifacts ranging from evaluation logs and benchmark cases to optimized prompts, attacker models, and reproduction triggers. These materials can support scientific scrutiny, but they can also lower the cost of misuse. External red teaming can itself create information hazards when it exposes novel jailbreaks or harmful-content techniques [68]. Recent offensive-security evaluations illustrate why this concern can become operationally relevant [69].

A useful way to characterize artifact sensitivity is by how directly an artifact enables exploitation, including its actionability, reusability, and the expertise or search effort required to use it. These dimensions are best viewed as an organizing lens rather than a formal risk score. The reviewed studies also suggest why file type alone is insufficient.

Qi et al. gated realistic harmful benchmark questions while reporting results on the public AdvBench benchmark, while offensive-security studies restricted prompts, source documents, or agent implementations and still reported the resulting evaluations [69–71]. Together, these practices suggest that sensitivity depends more on operational reusability than on artifact format alone.

The reviewed studies use markedly different release practices. Harmful evaluation sets have been placed behind manually reviewed access or withheld, while offensive-security studies have restricted prompts, implementation details, or target identities and disclosed sensitive findings directly to providers [69–71]. These practices vary along four main dimensions: content, access, timing, and disclosure process. Solaiman et al.'s staged release of GPT-2 provides an earlier precedent for conditioning availability on evolving risk assessment, but not a ready-made standard for red-team artifacts [72].

These restrictions need not eliminate scientific transparency. A study can remain auditable by documenting the threat model, experimental setup, evaluation procedure, and representative non-sensitive evidence even when prompts, source documents, target identities, or agent implementations are not public [69–71, 73]. This does not guarantee full reproducibility, but it preserves information needed to scrutinize the reported claim. Open-release benchmarks such as Cybench, HarmBench, and the red-team dataset of Ganguli et al. show that public artifacts can support inspection and standardized comparison when tasks, threat models, and evaluation protocols are explicitly scoped. These practices do not establish that the same release choice is appropriate for directly reusable exploit chains [6, 14, 74].

Based on these practices, we argue that reporting should preserve the evaluation trace from the threat model and attack-generation process through execution and evaluation to the observed failure. Relevant model or environment versions, evaluation criteria, and cost should be reported where they affect interpretation. If remediation is part of the study, the trace should also include the mitigation and a held-out regression result; an attack-discovery study need not make a remediation claim that it did not evaluate. The reviewed studies make different portions of the setup and evidence available [68–71]. When artifacts are restricted, authors should state clearly which experimental details and materials remain available for independent scrutiny. Shared rules for access rights, release timing, benchmark maintenance, and public accountability remain a field-level governance problem discussed in Section 9.6.

■ 9 Open Problems: Research Agenda

9.1 Coverage and Discovery Guarantees

Coverage in automated red teaming can be decomposed into four questions: **representation**, which defines the space to be covered; **identity**, which determines when two observations constitute the same failure; **discovery dynamics**, which measures how new failures accumulate with testing budget; and **stopping confidence**, which asks what finite testing implies about failures that remain unseen. These questions form a dependency chain rather than four independent evaluation criteria: discovery counts depend on the failure identity, identity is meaningful only relative to a representation, and a stopping claim requires a

discovery process whose sampling assumptions are understood.

- Representation: What is the denominator?

Coverage claims first require an explicit answer to “coverage of what?” Existing benchmarks largely adopt a closed-world representation. Harm-Bench and JailbreakBench define coverage over predefined harmful-behavior sets [14, 19], SALAD-Bench over a safety taxonomy [75], and HELM over scenario or metric spaces [56]. These choices support standardized comparison and repeatable evaluation within the selected spaces, but they do not establish whether failures outside those representations are covered. MultiMon provides a complementary open-world direction by discovering systematic and generalizable multimodal failure categories from observations [76]; however, the size and structure of the remaining failure space are still unspecified. The open problem is therefore not merely benchmark size. Existing evaluations largely support coverage claims within a predefined representation, whereas automated red teaming ultimately benefits from an open-world notion of coverage that can accommodate failure regions not specified in advance.

- Identity: What counts as a distinct failure?

Once the representation is chosen, coverage requires a stable unit of failure identity. Automated red teaming values diverse generated tests, and Perez et al. study scalable generation of varied red-team test cases across harm settings [7]. However, prompt diversity, attack diversity, and failure diversity are not equivalent. Different prompts, templates, strategies, or responses may instantiate the same failure, while similar observations may reveal different mechanisms. Failure identity may therefore be defined at several levels, including instance-level behavior, semantic harm category, interaction pattern, or underlying mechanism and context. These levels induce different deduplication units and therefore different coverage estimates. MultiMon is relevant here because it moves from individual failure instances toward systematic categories, linking validated examples to generalizable category descriptions [76]. Quality Diversity provides a useful analogy: distinctness is defined relative to behavior descriptors or niches [77]. It does not, by itself, constitute an established red-teaming solution. Without a stable identity rule, novelty counts and marginal discovery lack a well-defined unit.

- Discovery dynamics: How does marginal discovery evolve?

The identity problem also determines how diminishing returns should be interpreted. LLM-Fuzzer [78], Tree of Attacks with Pruning (TAP) [79], Prompt Automatic Iterative Refinement (PAIR) [80], and AutoDAN [81] characterize direct red-team search through query budgets, pruning and tree search, iterative refinement, population or generation budgets, and related efficiency measures. These studies primarily measure **search efficiency**, whereas coverage requires **discovery efficiency**: the marginal rate at which distinct failure modes are uncovered as the testing budget increases. A discovery curve can reveal declining marginal yield, but that decline is ambiguous. It may indicate that the search has depleted the failures reachable by the current strategy, or that the strategy is trapped in a local region of the failure space. Consequently, **search saturation and coverage saturation must be distinguished**. A discovery curve cannot support a coverage-

saturation claim without an account of how the search process shapes what remains reachable.

- Stopping confidence: When can testing stop?

Strong coverage claims require more than finite testing; they require a defensible statement about what may remain unseen. In the sources examined for this section, we did not identify a direct automated-red-teaming study that estimates such uncertainty or provides a stopping criterion for this purpose. Chao’s nonparametric estimator for the total number of classes in a population offers a statistical analogue: observed occupancy counts, especially low-frequency classes, can inform estimates of unobserved classes [82]. This is a transferability challenge rather than a plug-in substitution. An automated-red-teaming analogue would need observed distinct failure classes, an account of the adaptive search process that produced them, and a severity-aware uncertainty statement about unseen failures. Adaptive search changes the sampling distribution, repeated observations can be dependent, failure identities may be unstable, severity is heterogeneous, and models or environments may change. The deeper methodological goal is to identify assumptions under which automated red teaming can report not only what it found, but also a defensible uncertainty statement about what may remain undiscovered.

9.2 Judgment: Evaluation Reliability

Automated red teaming is only as valid as the judgments used to measure attack success. Empirical comparisons show that safety evaluators can assign substantially different labels to the same outputs. JailbreakBench reports pronounced variation in false-positive and false-negative rates across rule-based, general-purpose, and specialized judges, while StrongREJECT finds that evaluator choice can change both estimated jailbreak effectiveness and the ranking of attack methods [19, 83]. These discrepancies are especially consequential on adversarially searched outputs, whose distribution may differ substantially from the data on which evaluators were originally validated. A large-scale human audit found that judge accuracy varied with attack type, victim model, and harm category, in some tested slices approaching chance performance; correcting judge errors substantially changed the apparent success of several attacks [84]. A complementary audit found large recall differences among LLM judges and showed that surface-form wrapping or a refusal prefix could reverse their decisions even when the harmful content remained unchanged [85]. Evaluator error in automated red teaming is therefore also measurement error: it can alter reported ASR, comparisons among attacks, and conclusions about the robustness of target models.

The closed-loop nature of automated red teaming makes this problem more consequential than ordinary post-hoc scoring. In a static evaluation, a judge error primarily corrupts the reported result. In an automated search, judge outputs may instead control candidate selection, prompt refinement, pruning, or termination, allowing the search process to receive repeated feedback from the evaluator’s mistakes. Tree of Attacks, for example, places a judge inside an iterative attack tree, where the evaluator contributes to pruning and success determination; this illustrates architectural coupling between a fallible evaluator

and the search process without by itself establishing error amplification [79]. Direct attacks on LLM judges show that this channel is exploitable: optimization-based response injections can make a judge select attacker-controlled candidates with high success rates, including in an automated preference-annotation setting [86]. Red-team-specific evidence likewise indicates that judge-guided optimization and large-scale candidate selection can increase measured success by exploiting false positives without a corresponding increase in human-rated harmfulness [84]. Once evaluator outputs determine which candidates survive and how attacks are refined, weaknesses in the evaluator can become weaknesses in the search objective itself. This creates a reward-hacking-like failure mode in which search optimizes measured success rather than human-validated harmfulness.

A natural research direction is to decouple the evaluator used to guide search from the evaluator used for final reporting, and to audit the latter on samples drawn from the evolving attack distribution. JailbreakBench operationalizes one part of this separation by prohibiting submissions from optimizing against its leaderboard judge [19]. Selective-evaluation methods offer another building block: calibrated estimates of likely human agreement can determine which judgments are accepted automatically and which are escalated to a stronger judge [87]. For automated safety red teaming, the open question is which uncertainty signals are informative enough for selective validation. Calibration, evaluator disagreement, and adversarial stress tests are candidate escalation signals for routing ambiguous or high-risk cases to independent judges or targeted human review. The same problem becomes harder in multimodal or action-based settings, where a failure may depend on joint evidence rather than a single textual response [88]. The research objective is therefore not one universally trusted judge, but a validation process that prevents search-time optimization signals from silently becoming the final measure of safety.

9.3 Stateful and Long-Horizon Testing

We organize long-horizon red teaming around four linked questions: what state persists across the test, when history becomes the attack object, what outcome counts as failure, and what environment can execute and expose that outcome. State persistence and interaction horizon are distinct but interacting dimensions. The first concerns information that must remain available across steps, such as memory or retrieval, tool and application state, authority, prior actions, or externally supplied context; the second concerns how long dependent interaction must unfold before the safety outcome is determined. Memory and retrieval attacks illustrate the first dimension: AgentPoison poisons long-term memory or RAG knowledge so that a later trigger retrieves malicious state, while PoisonedRAG shows how corrupted retrieval stores can shape later answers [89, 90]. Agent-security benchmarks extend the same representational problem to tool outputs, application state, and externally supplied context [38, 91–93]. AgentPoison is also a useful boundary example because its poison-then-trigger structure is stateful without requiring an extended online interaction trajectory. Persistence determines what information the test must retain; horizon determines how deeply later decisions depend on earlier interaction.

When later attacker choices depend on earlier observations or state, the attack object moves from an isolated input to an interaction trajectory. MUSE provides a direct example by using tree search over multi-turn dialogue trajectories, so later attacker moves condition on conversation history and previous model responses [94]. Agent-security studies provide related examples of planning and state-conditioned interaction in settings involving tools, memory, and security objectives [38, 91]. Long-horizon testing is therefore better characterized by conditional dependence across steps than by raw turn count. Persistent state can create path dependence, while trajectory search actively exploits that dependence. Moving from single-step to trajectory-level testing consequently changes both the search object and the evaluation object.

This shift also changes the unit of evaluation. Trajectory-level red teaming changes not only what is searched, but also what is judged: a failure may be defined by actions, security properties, state transitions, or downstream consequences. AgentDojo and Agent Security Bench evaluate tool and security-property outcomes, while InjecAgent focuses on harm and private-data exfiltration in tool-integrated agents [38, 91, 92]. ToolEmu illustrates consequence-based evaluation for high-stakes tool use [44]. Other interactive environments evaluate end-of-conversation database state or execution-based OS and application state [95, 96]. Whether an executed action constitutes a security failure can depend on the agent’s authority or the applicable task policy [38, 95]. Existing studies thus provide examples of these outcome types, but the evaluation unit and success criterion must still be specified for each trajectory-level task.

Meaningful evaluation of failures whose preconditions or consequences depend on mutable tool, application, policy, or external state generally requires an environment that exposes the relevant transitions and consequences. Dynamic environments contribute executable actions, mutable external state, policy constraints, and observable downstream effects. AgentDojo and WebInject instantiate these requirements in adversarial tool or web-agent settings, while ToolEmu provides an emulated sandbox for evaluating severe tool-use consequences [38, 44, 97]. WebArena, OSWorld, and tau-bench provide complementary machinery for executable web tasks, real OS or application actions, and user-agent-tool interaction with observable final state [95, 96, 98]. Taken together, these strands point to a central open challenge: bringing explicit security objectives, mutable state transitions, adaptive adversarial trajectories, and auditable success criteria into the same evaluation setting.

9.4 Transfer Across Models, Representations, and Systems

A failure discovered on one target does not automatically generalize beyond that target. Transferability asks what, if anything, remains valid when the model, representation, or deployment changes. We distinguish three notions of transfer. *Artifact-level transfer* means that the same concrete object, such as a suffix or jailbreak prompt, remains effective on a new target. *Strategy-level transfer* means that the attack-generation or search principle remains useful, although target-specific artifacts may have to be regenerated. *Failure-level transfer* means

that the underlying safety failure remains meaningful and reproducible after the boundary changes. We organize these notions along three boundaries: changes to the model or alignment procedure; changes to a preserved intent's language, modality, encoding, or format; and changes to the deployed system, including application logic, system prompt, guardrails, policy, and orchestration. Failure-level transfer requires comparable source and target failure conditions, not just two high attack-success scores.

Across models, transfer is strongly source–target dependent. GCG showed that suffixes optimized on harmful behaviors and Vicuna models could elicit objectionable responses from several unseen open models and proprietary interfaces, while also reporting markedly lower success on some targets and signs that longer optimization could overfit the source models [99]. Subsequent methods reported stronger transfer in some settings without removing this target dependence. AutoDAN's automatically evolved, readable prompts transferred from several open-source models to GPT-3.5 more effectively than its GCG baseline, yet transfer was asymmetric across source–target pairs and remained low for the tested GPT-4 version [81]. AmpleGCG learned a generator over successful GCG candidates and reported transfer to several unseen targets, although its strongest closed-model results depended on the target version and evaluation protocol [100]. A broader study found poor and inconsistent transfer across 13 open models and repeated trigger optimizations, with substantially greater robustness among the tested preference-optimized models than fine-tuning-aligned models [101]. Artifact-level transfer can occur, but it varies substantially across source–target pairs and experimental configurations. It should therefore be reported as a directed relation rather than as an intrinsic label attached to a jailbreak.

The same harmful intent can produce different safety outcomes when its representation changes. In multilingual evaluation of ChatGPT and GPT-4, native-speaker translations of the same harmful queries produced materially different unsafe rates across nine non-English languages, with higher vulnerability in the tested low-resource group than in the high-resource group; multilingual jailbreak instructions further changed the observed safety behavior [102]. This is evidence of language-dependent robustness, not unchanged-artifact transfer from English. Con Instruction applies the same construction principle to separately optimized image and audio inputs across tested vision- and audio-language models, showing cross-modal generalization at the strategy level rather than direct reuse of one adversarial artifact [103]. English-only and text-only red teaming may therefore miss failures that emerge under alternative representations. Comparisons should preserve the risk intent, document the transformation, and use semantically comparable criteria; otherwise, translation quality or modality-specific judging can be mistaken for transfer.

At the system boundary, attack strategies may remain portable while the meaning of success changes. HouYi adapted a prompt-injection strategy and found at least one successful exploit scenario in 31 of 36 tested LLM-integrated applications, while its failures on other applications show that application logic and interfaces can interrupt portability [104]. Jailbreak-Zero generalized to safety policies unseen during

attacker fine-tuning, and selected attacks transferred across the tested victim families [105]. POEX makes the boundary sharper in embodied systems: text-centric jailbreak objectives could yield accepted or harmful-looking policies that were not executable, motivating separate acceptance and execution-success criteria [106]. A harmful textual response is not equivalent to harmful system behavior, and neither is equivalent to a harmful and executable policy. Transfer evaluation should therefore specify the source–target boundary, the object expected to transfer, and whether the risk objective and success condition are preserved or explicitly mapped. When the success condition changes, the result should not be counted as failure-level transfer without an explicit mapping between the two failure definitions.

9.5 Continual Red Teaming: Evaluation Drift

Standardized red-team evaluations make comparison possible by specifying both what is measured and how it is measured. HarmBench and JailbreakBench do this by defining harmful-behavior targets, threat models, artifacts, judges, attack settings, and reporting conventions, so that differences in measured attack success can be interpreted under shared evaluation conditions [14, 19]. Continual red teaming changes that setting. As new failures, capabilities, applications, and policies appear, continual evaluation creates pressure to update risk taxonomies, search policies, and regression suites. For longitudinal interpretation, three changes must be distinguished. *System drift* changes the model, application, guardrails, or configuration being tested. *Evaluation-target drift* changes the safety property, through a new harm definition, policy interpretation, failure condition, or risk category. *Evaluation-procedure drift* changes how the property is measured, including the case distribution, attack or search policy, judge or scoring rule, and budget or configuration. The same intended safety target can therefore be probed differently even when the tested system has not changed.

Measured vulnerability depends in part on how the system is probed. Standardized red-team benchmarks make this visible at the evaluation layer: generation settings, evaluator choices, attack choices, and resource accounting can change measured ASR even under shared harmful-behavior targets and benchmark conventions [14, 19]. At the adversarial exploration layer, h4rm3l expands the attack space through compositional transformations and synthesis, while Tree of Attacks varies branching, pruning, evaluator choice, depth, width, and query use [107, 108]. These choices affect which successful attacks are found and how efficiently they are found. ASR is therefore better interpreted as a conditional measurement of a target system under a specified evaluation configuration—including the target definition, case distribution, attack or search policy, judge, and budget—than as a system-intrinsic scalar. If those conditions change between two evaluations, a change in ASR cannot automatically be attributed to deterioration of the target system.

Recent work shows that safety evaluation itself can be adaptive. AgenticEval ingests a policy or regulatory framework, generates a test suite, evaluates a target model, and uses evaluation feedback to produce harder or more targeted cases over successive rounds [109]. In that setting, the target model and source framework can remain fixed while the

test content and procedure intensify, and the measured safety rate can fall as a result [109]. DAS provides complementary healthcare-domain evidence that dynamic adversarial auditing can reveal vulnerabilities missed by static benchmarks when agents mutate medically grounded cases across risk axes [110]. Adaptive evaluation can improve failure discovery by generating harder, more targeted, or broader probes, but that adaptation also changes the measurement context in which a score is obtained.

Once the system, target definition, or measurement procedure evolves, successive scores require an explicit comparability argument. Supporting benchmark-maintenance work shows that similar problems arise when evaluation sets evolve outside safety red teaming. Dynabench directly raises the problem of comparing results when a benchmark keeps changing and discusses retaining previous rounds together with high-quality static test sets as one response [111]. LiveBench provides a different precedent through benchmark refresh, versioned updates, and model re-runs [112]. What remains unresolved is how scores from evolving evaluations should be linked across versions and how observed drift should be attributed. One promising design is to combine an evolving discovery suite with a stable anchor set, explicit versioning of the system, target definition, and measurement configuration, and selective old/new cross-evaluation. A stable anchor cannot identify every source of drift, but it makes comparability a property to be tested rather than assumed.

9.6 Governance: Responsible Release

Governance becomes a research problem when red-team artifacts, evaluation access, and release decisions must work across organizations rather than within a single study. Section 8.3 considered how one team can restrict or disclose artifacts while preserving enough evidence to scrutinize a claim. At the field level, the unresolved question is who sets and documents those decisions when researchers, model providers, benchmark maintainers, and independent evaluators have different responsibilities. A review of public generative-AI red-teaming accounts found substantial variation in what findings and details were disclosed, alongside no standardized reporting procedure [113]. External red-teaming practice already treats access level as a design choice, while staged release provides a precedent for varying availability as risk information develops [68, 72]. Coordinated Flaw Disclosure is a concrete proposal for adapting vulnerability-disclosure ideas to AI flaws, including mechanisms for verification and adjudication [114]; it is not, however, an established protocol for automated red teaming. The research task is therefore to specify shared, risk-sensitive procedures that record decision rights, access tier, release timing, disclosure recipient, and claim-verifying evidence for sensitive artifacts. The aim is not a universal release rule, but an auditable process for balancing controlled access, scientific scrutiny, and coordinated disclosure.

Shared evaluation artifacts need stewardship as well as access control. Automated red teaming depends on behavior sets, attack libraries, judges, regression cases, and execution environments that change as models and risks change. Section 9.5 treated the resulting comparability problem as measurement drift; the governance question is who

maintains these evolving artifacts and records their history for other users. Recent lifecycle-oriented work on medical LLM benchmarks treats documentation, openness, and governance as a distinct phase, including long-term maintenance responsibility, versioning plans, and feedback channels [115]. Although that study concerns a different domain, it identifies a stewardship problem directly relevant to shared safety evaluations. Dynabench illustrates dynamic benchmark creation, and LiveBench illustrates regular benchmark refresh and versioned model evaluation [111, 112]. These are not governance models for automated red teaming, but they show why an evolving benchmark cannot be treated as a static public object. Community protocols should assign maintenance ownership and require version identifiers, changelogs, provenance, and policies for retaining or deprecating cases, judges, and attack artifacts. They should also specify how stable anchor sets coexist with renewed suites. Such stewardship would not resolve the attribution problem in Section 9.5, but it would make the history and ownership of a shared evaluation artifact inspectable.

High-severity findings create a third coordination problem: how a discovery moves from an experimental result to a responsible response. In an offensive-security setting, Zhu et al. withheld code and prompts and disclosed findings through a provider's responsible-disclosure program [71]. Feffer et al.'s review and the Coordinated Flaw Disclosure proposal both make clear that such reporting remains fragmented rather than governed by a field-wide operational standard [113, 114]. Automated red teaming therefore needs an escalation path for findings whose severity, exploitability, or affected deployment warrants action beyond publication. Open questions include how severity is assessed, which providers or maintainers are notified, what reproduction evidence and remediation status are recorded, when public disclosure is appropriate, and which records remain available for external accountability. The aim is not to turn every red-team result into a public incident report, but to ensure that serious findings have an auditable handoff and follow-up path. As automated red teaming produces more shared evidence and potentially reusable attack artifacts, its technical progress increasingly depends on how that evidence is maintained, disclosed, and acted upon collectively.

10 Conclusion

Automated red teaming should be evaluated as a closed-loop safety-evaluation system rather than as a collection of isolated jailbreak techniques. A useful survey must therefore connect risk specification, generation, search, interaction, judgment, coverage, mitigation, and responsible operational practice across generative-model targets.

Acknowledgements

The authors thank the researchers and practitioners who develop public safety benchmarks, red-teaming tools, and responsible disclosure practices.

Competing Interest

The authors declare that they have no competing interests or financial conflicts to disclose.

■ A Detailed Survey Protocol

A.1 Search Strings: Sources

Add the final database list, search strings, date range, and screening flow here.

A.2 Paper Annotation: Evidence Table

Add the complete coding fields and examples used to classify each paper.

■ B Extended Taxonomy Tables

Add tables comparing papers by target system, threat model, generation strategy, interaction horizon, evaluator, metrics, and release status.

■ C Evaluation Checklist

Add a checklist that reviewers can use to assess whether an automated red-teaming result is reproducible, valid, sufficiently broad, and responsibly reported.

■ References

- [1] Lin L, Mu H, Zhai Z, Wang M, Wang Y, Wang R, Gao J, Zhang Y, Che W, Baldwin T, Han X, Li H. Against the achilles' heel: A survey on red teaming for generative models. *Journal of Artificial Intelligence Research*, 2025, 82: 687–775
- [2] Das B C, Amini M H, Wu Y. Security and privacy challenges of large language models: A survey. *ACM Computing Surveys*, 2025, 57(6): 1–39
- [3] Jabbar M S, Al-Azani S, Alotaibi A, Ahmed M. Red teaming large language models: A comprehensive review and critical analysis. *Information Processing and Management*, 2025, 62(6): 104239
- [4] Purpura A, Wadhwa S, Zymet J, Gupta A, Luo A, Rad M K, Shinde S, Sorower M S. Building safe genai applications: An end-to-end overview of red teaming for large language models. In: *Proceedings of the 5th Workshop on Trustworthy NLP*. 2025, 335–350
- [5] Liu X, Zhu Y, Lan Y, Yang C, Qiao Y. Safety of multimodal large language models on images and texts, 2024
- [6] Ganguli D, Lovitt L, Kernion J, Askell A, Bai Y, Kadavath S, Mann B, Perez E, Schiefer N, Ndousse K, Jones A, Bowman S, Chen A, Conerly T, DasSarma N, Drain D, Elhage N, El-Showk S, Fort S, Hatfield-Dodds Z, Henighan T, Hernandez D, Hume T, Jacobson J, Johnston S, Kravec S, Olsson C, Ringer S, Tran-Johnson E, Amodei D, Brown T, Joseph N, McCandlish S, Olah C, Kaplan J, Clark J. Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned. *arXiv preprint arXiv:2209.07858*, 2022
- [7] Perez E, Huang S, Song F, Cai T, Ring R, Aslanides J, Glaese A, McAleese N, Irving G. Red teaming language models with language models. In: *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. 2022, 3419–3448
- [8] Ge S, Zhou C, Hou R, Khabsa M, Wang Y C, Wang Q, Han J, Mao Y. MART: Improving LLM safety with multi-round automatic red-teaming. In: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 2024, 1927–1937
- [9] Lee S, Kim M, Cherif L, Dobre D, Lee J, Hwang S J, Kawaguchi K, Gidel G, Bengio Y, Malkin N, Jain M. Learning diverse attacks on large language models for robust red-teaming and safety tuning. In: *International Conference on Learning Representations*. 2025
- [10] Zhao A, Xu Q, Lin M, Wang S, Liu Y J, Zheng Z, Huang G. Diver-ct: Diversity-enhanced red teaming large language model assistants with relaxing constraints. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. 2025, 26021–26030
- [11] Bianchi F, Suzgun M, Attanasio G, Röttger P, Jurafsky D, Hashimoto T, Zou J Y. Safety-tuned llamas: Lessons from improving the safety of large language models that follow instructions. In: *International Conference on Learning Representations*. 2024
- [12] Pan L, Tong Y, Zhang X, Zhang X, Zhou J, Chu Z. Understanding and mitigating overrefusal in LLMs from an unveiling perspective of safety decision boundary. In: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 2025, 21057–21075
- [13] Röttger P, Kirk H, Vidgen B, Attanasio G, Bianchi F, Hovy D. XSTest: A test suite for identifying exaggerated safety behaviours in large language models. In: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. 2024, 5377–5400
- [14] Mazeika M, Phan L, Yin X, Zou A, Wang Z, Mu N, Sakhaee E, Li N, Basart S, Li B, Forsyth D, Hendrycks D. HarmBench: A standardized evaluation framework for automated red teaming and robust refusal. In: *Proceedings of the 41st International Conference on Machine Learning*. 2024, 35181–35224
- [15] Wei A, Haghtalab N, Steinhardt J. Jailbroken: How does LLM safety training fail? In: *Advances in Neural Information Processing Systems*. 2023
- [16] Andriushchenko M, Croce F, Flammarion N. Jailbreaking leading safety-aligned llms with simple adaptive attacks. In: *International Conference on Learning Representations*. 2025
- [17] Andriushchenko M, Flammarion N. Does refusal training in LLMs generalize to the past tense? In: *International Conference on Learning Representations*. 2025
- [18] Wang H, Qin Z, Shen L, Wang X, Tao D, Cheng M. Safety reasoning with guidelines. In: *Proceedings of the 42nd International Conference on Machine Learning*. 2025, 64084–64108
- [19] Chao P, DeBenedetti E, Robey A, Andriushchenko M, Croce F, Schwag V, Dobriban E, Flammarion N, Pappas G J, Tramèr F, Hassani H, Wong E. Jailbreakbench: An open robustness benchmark for jailbreaking large language models. In: *Advances in Neural Information Processing Systems*. 2024
- [20] Han S, Rao K, Ettinger A, Jiang L, Lin B Y, Lambert N, Choi Y, Dziri N. Wildguard: Open one-stop moderation tools for safety risks, jailbreaks, and refusals of LLMs. In: *Advances in Neural Information Processing Systems*. 2024
- [21] Inan H, Upasani K, Chi J, Rungta R, Iyer K, Mao Y, Tontchev M, Hu Q, Fuller B, Testuggine D, Khabsa M. Llama guard: LLM-based input-output safeguard for human-AI conversations. *arXiv preprint arXiv:2312.06674*, 2023

- [22] Zeng W, Liu Y, Mullins R, Peran L, Fernandez J, Harkous H, Narasimhan K, Proud D, Kumar P, Radharapu B, Sturman O, Wahltinez O. Shieldgemma: Generative AI content moderation based on gemma. arXiv preprint arXiv:2407.21772, 2024
- [23] Rebedea T, Dinu R, Sreedhar M N, Parisien C, Cohen J. NeMo guardrails: A toolkit for controllable and safe LLM applications with programmable rails. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations. 2023, 431–445
- [24] Liu H, Huang H, Gu X, Wang H, Wang Y. On calibration of LLM-based guard models for reliable content moderation. In: International Conference on Learning Representations. 2025
- [25] Zhang Y, Ding L, Zhang L, Tao D. Intention analysis makes LLMs a good jailbreak defender. In: Proceedings of the 31st International Conference on Computational Linguistics. 2025, 2947–2968
- [26] Xu Z, Jiang F, Niu L, Jia J, Lin B Y, Poovendran R. SafeDecoding: Defending against jailbreak attacks via safety-aware decoding. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2024, 5587–5605
- [27] Shen G, Zhao D, Dong Y, He X, Zeng Y. Jailbreak antidote: Runtime safety-utility balance via sparse representation adjustment in large language models. In: International Conference on Learning Representations. 2025
- [28] Ghosal S S, Chakraborty S, Singh V, Guan T, Wang M, Beirami A, Huang F, Velasquez A, Manocha D, Bedi A S. Immune: Improving safety against jailbreaks in multi-modal LLMs via inference-time alignment. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2025, 25038–25049
- [29] Zhu J, Yan L, Wang S, Yin D, Sha L. Reasoning-to-defend: Safety-aware reasoning can defend large language models from jailbreaking. In: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing. 2025, 29343–29361
- [30] Xu Y, Qi X, Qin Z, Wang W. Cross-modality information check for detecting jailbreaking in multimodal large language models. In: Findings of the Association for Computational Linguistics: EMNLP 2024. 2024, 13715–13726
- [31] Chi J, Karn U, Zhan H, Smith E, Rando J, Zhang Y, Plawiak K, Delpierre Coudert Z, Upasani K, Pasupuleti M. Llama guard 3 vision: Safeguarding human-AI image understanding conversations. arXiv preprint arXiv:2411.10414, 2024
- [32] Verma S, Hines K, Bilmes J, Siska C, Zettlemoyer L, Gonen H, Singh C. MULTIGUARD: An efficient approach for AI safety moderation across languages and modalities. In: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing. 2025, 16173–16187
- [33] Yoon J, Yu S, Patil V R, Yao H, Bansal M. Safree: Training-free and adaptive guard for safe text-to-image and video generation. In: International Conference on Learning Representations. 2025
- [34] Liang S, Liu J, Zhai J, Fang T, Tu R, Liu A, Cao X, Tao D. T2vshield: Model-agnostic jailbreak defense for text-to-video models. arXiv preprint arXiv:2504.15512, 2025
- [35] He J, Hou G, Li H, Huang Z, Chen K, Yu Y, Jiang W, Xu G, Zhang T. Tear: Temporal-aware automated red-teaming for text-to-video models. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). June 2026, 41–50
- [36] Yang Y, Zhang X, Han Z, Wang S, Zhuang J, Jin Z, Shao J, Sun G, Zhang C. Speech-audio compositional attacks on multimodal LLMs and their mitigation with SALMONN-guard. arXiv preprint arXiv:2511.10222, 2026
- [37] Kang M, Fang C, Li B. Audioguard: Toward comprehensive audio safety protection across diverse threat models. arXiv preprint arXiv:2604.08867, 2026
- [38] Debenedetti E, Zhang J, Balunović M, Beurer-Kellner L, Fischer M, Tramèr F. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. In: Advances in Neural Information Processing Systems 37. 2024. Datasets and Benchmarks Track
- [39] Debenedetti E, Shumailov I, Fan T, Hayes J, Carlini N, Fabian D, Kern C, Shi C, Terzis A, Tramèr F. Defeating prompt injections by design. arXiv preprint arXiv:2503.18813, 2025
- [40] Wu Y, Roesner F, Kohno T, Zhang N, Iqbal U. IsolateGPT: An execution isolation architecture for LLM-based agentic systems. In: Proceedings of the Network and Distributed System Security Symposium. 2025
- [41] Jia F, Wu T, Qin X, Squicciarini A. The task shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics. 2025, 29680–29697
- [42] Mou Y, Xue Z, Li L, Liu P, Zhang S, Ye W, Shao J. Toolsafe: Enhancing tool invocation safety of LLM-based agents via proactive step-level guardrail and feedback. In: Findings of the Association for Computational Linguistics: ACL 2026. 2026, 37125–37153
- [43] Yan Z, Weng J, Chen C, Peng D, Qin E, Guan J, Liu J, Yu Q, Yuan Y, Meng F, Che C, Hu M. Do coding agents understand least-privilege authorization? arXiv preprint arXiv:2605.14859, 2026
- [44] Ruan Y, Dong H, Wang A, Pitis S, Zhou Y, Ba J, Dubois Y, Maddison C, Hashimoto T. Identifying the risks of LM agents with an LM-emulated sandbox. In: International Conference on Learning Representations. 2024
- [45] Chen Z, Liu H, Xu D, Dong D, Li J, Pu B, Zhai J. Cordon: Semantic transactions for tool-using LLM agents. arXiv preprint arXiv:2606.17573, 2026
- [46] OpenAI . Gpt-4 system card, 2023
- [47] Zhou A, Wu K, Pinto F, Chen Z, Zeng Y, Yang Y, Yang S, Koyejo S, Zou J Y, Li B. Autoreddteamer: Autonomous red teaming with lifelong attack integration. In: Advances in Neural Information Processing Systems. 2025
- [48] Zhang J, Elgohary A, Wang X, Iftekhhar A S M, Magooda A, Van Durme B, Khashabi D, Jackson K. Jailbreak distillation: Renewable safety benchmarking. In: Findings of the Association for Computational Linguistics: EMNLP 2025. 2025, 25066–25089
- [49] Derczynski L, Galinkin E, Martin J, Majumdar S, Inie N. garak: A framework for security probing large language models, 2024
- [50] Lopez Munoz G D, Minnich A J, Lutz R, Lundeen R, Dheekonda

- R S R, Chikanov N, Jagdagdorj B E, Pouliot M, Chawla S, Maxwell W, Bullwinkel B, Pratt K, Gruyter d J, Siska C, Bryan P, Westerhoff T, Kawaguchi C, Seifert C, Kumar R S S, Zunger Y. PyRIT: A framework for security risk identification and red teaming in generative ai system, 2024
- [51] AI Security Institute, UK . Inspect AI: Framework for large language model evaluations, 2024. Official software documentation. Accessed 2026-08-09
- [52] promptfoo . promptfoo: Llm evals and red teaming. Official software documentation. Accessed 2026-08-09
- [53] Meta Purple Llama Contributors . CybersecurityBenchmarks. Official software repository documentation. Accessed 2026-08-09
- [54] Liu X, Zhu Y, Gu J, Lan Y, Yang C, Qiao Y. MM-SafetyBench: A benchmark for safety evaluation of multimodal large language models. In: Proceedings of the European Conference on Computer Vision (ECCV). 2024
- [55] OpenAI . OpenAI Evals. Official software repository. Accessed 2026-08-09
- [56] Liang P, Bommasani R, Lee T, Tsipras D, Soylu D, Yasunaga M, Zhang Y, Narayanan D, Wu Y, Kumar A, Newman B, Yuan B, Yan B, Zhang C, Cosgrove C, Manning C D, Re C, Acosta-Navas D, Hudson D A, Zelikman E, Durmus E, Ladhak F, Rong F, Ren H, Yao H, Wang J, Santhanam K, Orr L, Zheng L, Yuksekgonul M, Suzgun M, Kim N, Guha N, Chatterji N S, Khattab O, Henderson P, Huang Q, Chi R A, Xie S M, Santurkar S, Ganguli S, Hashimoto T, Icard T, Zhang T, Chaudhary V, Wang W, Li X, Mai Y, Zhang Y, Koreeda Y. Holistic evaluation of language models. Transactions on Machine Learning Research, 2023
- [57] Quaye J, Parrish A, Inel O, Rastogi C, Kirk H R, Kahng M, Van Liemt E, Bartolo M, Tsang J, White J, Clement N, Mosquera R, Ciro J, Reddi V J, Aroyo L. Adversarial nibbler: An open red-teaming method for identifying diverse harms in text-to-image generation. In: Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency. 2024, 388–406
- [58] Rastogi C, Bhutani M, Kahng M, Muhammad S H, Razumovskaia E, Suresh P, Ahmad I S, Kalia C, Mahomed Y, Maji M, Lee M, Parrish A, Quaye J, Reddi V J, Verma A, Aroyo L. Going PLACES: Participatory localized red teaming for text-to-image safety in the global south. In: Proceedings of the 2026 ACM Conference on Fairness, Accountability, and Transparency. 2026, 2608–2642
- [59] Weidinger L, Mellor J F J, Pegueroles B G, Marchal N, Kumar R, Lum K, Akbulut C, Diaz M, Bergman A S, Rodriguez M D, Rieser V, Isaac W. STAR: SocioTechnical approach to red teaming language models. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. 2024, 21516–21532
- [60] Chiu Y Y, Jiang L, Lin B Y, Park C Y, Li S S, Ravi S, Bhatia M, Antoniak M, Tsvetkov Y, Shwartz V, Choi Y. CulturalBench: A robust, diverse and challenging benchmark for measuring LMs' cultural knowledge through human-AI red-teaming. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2025, 25663–25701
- [61] Kim S, Jo Y, Na S, Lim H, Lee E, Choi Y M, Cho S, Hong H. Dark and bright side of participatory red-teaming with targets of stereotyping for eliciting harmful behaviors from large language models. arXiv preprint arXiv:2602.19124, 2026
- [62] Radharapu B, Robinson K, Aroyo L, Lahoti P. AART: AI-assisted red-teaming with diverse data generation for new LLM-powered applications. In: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: Industry Track. 2023, 380–395
- [63] Quaye J, Parrish A, Rastogi C, Kahng M, Inel O, Aroyo L, Reddi V J. Bridging the scale gap: Augmenting human red-teaming to uncover latent risks in T2I models. In: Proceedings of the 2026 ACM Conference on Fairness, Accountability, and Transparency. 2026, 7422–7449
- [64] Alvarado Garcia A, Wan R, Oguine O C, Badillo-Urquiola K. Red teaming LLMs as socio-technical practice: From exploration and data creation to evaluation. In: Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems. 2026
- [65] Dinan E, Humeau S, Chintagunta B, Weston J. Build it break it fix it for dialogue safety: Robustness from adversarial human attack. In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing. 2019, 4537–4546
- [66] Aroyo L, Taylor A S, Díaz M, Homan C M, Parrish A, Serapio-García G, Prabhakaran V, Wang D. DICES dataset: Diversity in conversational AI evaluation for safety. In: Advances in Neural Information Processing Systems. 2023. Datasets and Benchmarks Track
- [67] Davani A, Díaz M, Baker D, Prabhakaran V. D3CODE: Disentangling disagreements in data across cultures on offensiveness detection and evaluation. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. 2024, 18511–18526
- [68] Ahmad L, Agarwal S, Lampe M, Mishkin P. Openai's approach to external red teaming for ai models and systems, 2025
- [69] Fang R, Bindu R, Gupta A, Zhan Q, Kang D. Llm agents can autonomously hack websites, 2024
- [70] Qi X, Zeng Y, Xie T, Chen P Y, Jia R, Mittal P, Henderson P. Fine-tuning aligned language models compromises safety, even when users do not intend to!, 2023
- [71] Zhu Y, Kellermann A, Gupta A, Li P, Fang R, Bindu R, Kang D. Teams of llm agents can exploit zero-day vulnerabilities, 2025
- [72] Solaiman I, Brundage M, Clark J, Askell A, Herbert-Voss A, Wu J, Radford A, Krueger G, Kim J W, Kreps S, McCain M, Newhouse A, Blazakis J, McGuffie K, Wang J. Release strategies and the social impacts of language models, 2019
- [73] Fang R, Bindu R, Gupta A, Kang D. Llm agents can autonomously exploit one-day vulnerabilities, 2024
- [74] Zhang A K, Perry N, Dulepet R, Ji J, Menders C, Lin J W, Jones E, Hussein G, Liu S, Jasper D, Peetathawatchai P, Sivashankar V, Zamoshchin D, Glikbarg L, Askaryar D, Yang H, Zhang A, Alluri R, Tran N, Sangpisit R, Yiorkadjis P, Osele K, Raghupathi G, Boneh D, Ho D E, Liang P. Cybench: A framework for evaluating cybersecurity capabilities and risks of language models, 2025

- [75] Li L, Dong B, Wang R, Hu X, Zuo W, Lin D, Qiao Y, Shao J. SALAD-bench: A hierarchical and comprehensive safety benchmark for large language models. In: Findings of the Association for Computational Linguistics: ACL 2024. 2024, 3923–3954
- [76] Tong S, Jones E, Steinhardt J. Mass-producing failures of multimodal systems with language models. In: Advances in Neural Information Processing Systems. 2023, 29292–29322
- [77] Pugh J K, Soros L B, Stanley K O. Quality diversity: A new frontier for evolutionary computation. *Frontiers in Robotics and AI*, 2016, 3
- [78] Yu J, Lin X, Yu Z, Xing X. LLM-Fuzzer: Scaling assessment of large language model jailbreaks. In: 33rd USENIX Security Symposium (USENIX Security 24). 2024, 4657–4674
- [79] Mehrotra A, Zampetakis M, Kassianik P, Nelson B, Anderson H, Singer Y, Karbasi A. Tree of attacks: Jailbreaking black-box llms automatically. In: Advances in Neural Information Processing Systems. 2024, 61065–61105
- [80] Chao P, Robey A, Dobriban E, Hassani H, Pappas G J, Wong E. Jailbreaking black box large language models in twenty queries. In: 2025 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML). 2025, 23–42
- [81] Liu X, Xu N, Chen M, Xiao C. AutoDAN: Generating stealthy jailbreak prompts on aligned large language models. In: International Conference on Learning Representations. 2024, 56174–56194
- [82] Chao A. Nonparametric estimation of the number of classes in a population. *Scandinavian Journal of Statistics*, 1984, 11(4): 265–270
- [83] Souly A, Lu Q, Bowen D, Trinh T, Hsieh E, Pandey S, Abbeel P, Svegliato J, Emmons S, Watkins O, Toyer S. A strongreject for empty jailbreaks. In: Advances in Neural Information Processing Systems. 2024
- [84] Schwinn L, Ladenburger M, Beyer T, Mofakhami M, Gidel G, G'unnemann S. A coin flip for safety: Llm judges fail to reliably measure adversarial robustness. *arXiv preprint arXiv:2603.06594*, 2026
- [85] Gao Y. How reliable is your jailbreak judge? calibration and adversarial robustness of automated asr scoring. *arXiv preprint arXiv:2606.25487*, 2026
- [86] Shi J, Yuan Z, Liu Y, Huang Y, Zhou P, Sun L, Gong N Z. Optimization-based prompt injection attack to llm-as-a-judge. In: Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security. 2024, 660–674
- [87] Jung J, Brahman F, Choi Y. Trust or escalate: Llm judges with provable guarantees for human agreement. *arXiv preprint arXiv:2407.18370*, 2024
- [88] Li C, Li L, Shao J. Treeteaming: Autonomous red-teaming of vision-language models via hierarchical strategy exploration. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2026, 37684–37693
- [89] Chen Z, Xiang Z, Xiao C, Song D, Li B. AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases. In: Advances in Neural Information Processing Systems. 2024
- [90] Zou W, Geng R, Wang B, Jia J. PoisonedRAG: Knowledge corruption attacks to retrieval-augmented generation of large language models. In: 34th USENIX Security Symposium (USENIX Security 25). August 2025, 3827–3844
- [91] Zhang H, Huang J, Mei K, Yao Y, Wang Z, Zhan C, Wang H, Zhang Y. Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents. In: International Conference on Learning Representations. 2025
- [92] Zhan Q, Liang Z, Ying Z, Kang D. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In: Findings of the Association for Computational Linguistics: ACL 2024. 2024, 10471–10506
- [93] Abdelnabi S, Greshake K, Mishra S, Endres C, Holz T, Fritz M. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security. 2023, 79–90
- [94] Yan S, Zeng L, Wu X, Han C, Zhang K, Peng C, Cao X, Cai X, Guo C. MUSE: MCTS-driven red teaming framework for enhanced multi-turn dialogue safety in large language models. In: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing. 2025, 21282–21303
- [95] Yao S, Shinn N, Razavi P, Narasimhan K. τ -bench: A benchmark for Tool-Agent-User interaction in real-world domains. In: International Conference on Learning Representations. 2025
- [96] Xie T, Zhang D, Chen J, Li X, Zhao S, Cao R, Hua T J, Cheng Z, Shin D, Lei F, Liu Y, Xu Y, Zhou S, Savarese S, Xiong C, Zhong V, Yu T. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In: Advances in Neural Information Processing Systems. 2024
- [97] Wang X, Bloch J, Shao Z, Hu Y, Zhou S, Gong N Z. WebInject: Prompt injection attack to web agents. In: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing. 2025, 2010–2030
- [98] Zhou S, Xu F F, Zhu H, Zhou X, Lo R, Sridhar A, Cheng X, Ou T, Bisk Y, Fried D, Alon U, Neubig G. WebArena: A realistic web environment for building autonomous agents. In: International Conference on Learning Representations. 2024
- [99] Zou A, Wang Z, Carlini N, Nasr M, Kolter J Z, Fredrikson M. Universal and transferable adversarial attacks on aligned language models, 2023
- [100] Liao Z, Sun H. AmpleGCG: Learning a universal and transferable generative model of adversarial suffixes for jailbreaking both open and closed LLMs. In: First Conference on Language Modeling. 2024
- [101] Meade N, Patel A, Reddy S. Investigating adversarial trigger transfer in large language models. *Transactions of the Association for Computational Linguistics*, 2025, 13: 953–979
- [102] Deng Y, Zhang W, Pan S J, Bing L. Multilingual jailbreak challenges in large language models. In: The Twelfth International Conference on Learning Representations. 2024
- [103] Geng J, Tran T T, Nakov P, Gurevych I. Con instruction: Universal jailbreaking of multimodal large language models via non-textual

- modalities. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2025, 2917–2933
- [104] Liu Y, Deng G, Li Y, Wang K, Wang Z, Wang X, Zhang T, Liu Y, Wang H, Zheng Y, Zhang L Y, Liu Y. Prompt injection attack against LLM-integrated applications, 2023
- [105] Hu K, Aggarwal A, Khodabandeh M, Zhang D, Hsin E, Chen L, Jain A, Fredrikson M, Bharadwaj A. Jailbreak-zero: A path to Pareto optimal red teaming for large language models. In: Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2026, 46720–46746
- [106] Lu X, Huang Z, Li X, Zhang C, Ji X, Xu W. POEX: Towards policy executable jailbreak attacks against the LLM-based robots, 2025
- [107] Doumbouya M K B, Nandi A, Poesia G, Ghilardi D, Goldie A, Bianchi F, Jurafsky D, Manning C. h4rm3l: A language for composable jailbreak attack synthesis. In: Yue Y, Garg A, Peng N, Sha F, Yu R, eds, International Conference on Learning Representations. 2025, 57253–57286
- [108] Mehrotra A, Zampetakis M, Kassianik P, Nelson B, Anderson H, Singer Y, Karbasi A. Tree of attacks: Jailbreaking black-box LLMs automatically. In: Globerson A, Mackey L, Belgrave D, Fan A, Paquet U, Tomczak J, Zhang C, eds, Advances in Neural Information Processing Systems. 2024, 61065–61105
- [109] Wang Y, Wang X, Yao Y, Li X, Yang X, Teng Y, Ma X, Wang Y. AgenticEval: Toward agentic and self-evolving safety evaluation of large language models. In: Findings of the Association for Computational Linguistics: ACL 2026. July 2026, 14789–14808
- [110] Pan J, Jian B, Hager P, Zhang Y, Liu C, Jungmann F, Li H B, Canisius J, You C, Wu J, Zhu J, Liu F, Liu Y, Bubeck N, Knolle M, Chen C, Wachinger C, Gong Z, Ouyang C, Kaissis G, Wiestler B, Rueckert D. Addressing benchmarking gaps in large language models for health and medicine with dynamic red-teaming. *Nature Health*, 2026
- [111] Kiela D, Bartolo M, Nie Y, Kaushik D, Geiger A, Wu Z, Vidgen B, Prasad G, Singh A, Ringshia P, Ma Z, Thrush T, Riedel S, Waseem Z, Stenetorp P, Jia R, Bansal M, Potts C, Williams A. Dynabench: Rethinking benchmarking in NLP. In: Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. June 2021, 4110–4124
- [112] White C, Dooley S, Roberts M, Pal A, Feuer B, Jain S, Shwartz-Ziv R, Jain N, Saifullah K, Dey S, Shubh-Agrawal, Sandha S, Naidu S, Hegde C, LeCun Y, Goldstein T, Neiswanger W, Goldblum M. LiveBench: A challenging, contamination-limited LLM benchmark. In: Yue Y, Garg A, Peng N, Sha F, Yu R, eds, International Conference on Learning Representations. 2025, 91595–91631
- [113] Feffer M, Sinha A, Deng W H, Lipton Z C, Heidari H. Red-teaming for generative AI: Silver bullet or security theater? *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society*, 2024, 7: 421–437
- [114] Cattell S, Ghosh A, Kaffee L A. Coordinated flaw disclosure for AI: Beyond security vulnerabilities. *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society*, 2024, 7: 267–280
- [115] Wang W, Ma Z, Yu G, Cheung Y F, Ding M, Liu J, Chen W, Shen L. Beyond the leaderboard: Rethinking medical benchmarks for large language models. In: Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). July 2026, 43078–43123